# Principal Component Analysis II

Sean Hellingman ©

Multivariate Statistics for Applied Data Science (ADSC 4050)

*shellingman@tru.ca*

Winter 2026

# Topics

# Recap

- Last time we covered **PCA I**:
  - Procedure.
  - Loadings.
  - Scores.

# Introduction

- May use PCA to reduce dimensionality when there are many correlated variables.
  - Allowing us to focus on the first few dimensions and still see broad patterns in our data.
- *More effective ways to implement this method.*
- How many principal components are needed?

# Base R Implementation

- We can use the `princomp()` function from base R (stats).
  - In Python.

- Usage:

```
Data.PCA <- princomp(data,cor = TRUE)

summary(Data.PCA, loadings = TRUE, cutoff = 0)
plot(Data.PCA) - Scree plot
```

- Note: If you set `cor=FALSE` your data should be scaled.

# Example 1

- Import the *Basketball.csv* dataset to do the following:
  1. Use the `princomp()` function to conduct PCA on the numeric variables of the dataset.
  2. Summarize the results.
  3. Generate a scree plot.

- What do you notice?

# h2o Package

- We can also use the h2o package for dimension reduction.
  - There is also a Python version and in general this package automates scaling and cleaning processes.
- Usage:

`h2o.no_progress()` - *turn off progress bars*
`h2o.init(max_mem_size = "5g")` - *connect to H2O instance*

`data.h2o <- as.h2o(data)` - *convert data*

```
my_pca <- h2o.prcomp(
 training_frame = data.h2o,
 pca_method = "GramSVD",
 k = ncol(data.h2o),
 transform = "STANDARDIZE",
 impute_missing = TRUE,
 max_runtime_secs = 1000)
```

## Some h2o PCA Arguments

- `pca_method`: Method used to compute principal components.
  - `"GramSVD"`: Best for mostly numeric data.
  - `"GLRM"`: Recommended when categorical variables are present.
- `k`: Number of principal components to compute.
  - Typically set to $p$.
- `transform`: Specifies whether the data should be standardized before PCA.
- `impute_missing`: If `TRUE`, missing values are replaced with the column mean.
- `max_runtime_secs`: Maximum training time (seconds).
  - *Useful for limiting runtime on large datasets.*

# Illustrative Example 1

1. Run the provided code to conduct PCA on the *iris* dataset.
2. Which features contribute most to the first PC?
   - What about the second PC?
3. How do the different features contribute to PC1 and PC2?

# Example 2

- Import the *Basketball.csv* dataset to do the following:
  1. Use the h2o package to conduct PCA on the numeric variables of the dataset.
  2. Which features contribute most to the first PC?
     - What about the second PC?
  3. How do the different features contribute to PC1 and PC2?

- What do you notice?

# Number of Principal Components

# Number of Principal Components

- If we use all of the principal components, we have just rotated our data and not actually reduced the dimensionality.
- To reduce the dimensionality, we will only focus on a few *important* PCs.
- There are a few different approaches that may yield different results.
  1. Scree plots.
  2. As many eigenvalues as necessary to account for 75% or 90% of the variation.
  3. Eigenvalue criterion.

# (1) Scree Plots

- A **scree plot** shows the proportion of variance explained by each principal component, with the PCs plotted in order.

- We can make selections off of the following criteria:
  - Look for a sharp bend or change in slope in the proportion of variance explained.
  - Focus on those principal components that explain more variation than average.
  - Focus on all principal components that explain more variation than *expected*.

# Scree Plots in R

- Base R:

```
eigen.prop <- Data.PCA$sdev / sum(Data.PCA$sdev)
barplot(eigen.prop)
abline(h = mean(eigen.prop), col = "red") - Add average line
```

- h2o in R:

```
data.frame(
 PC = my_pca@model$importance %>% seq_along,
 PVE = my_pca@model$importance %>% .[2,] %>% unlist())
%>%
 ggplot(aes(PC, PVE, group = 1, label = PC)) +
 geom_point() +
 geom_line() +
 geom_text(nudge_y = -.02)
```

# Illustrative Example 2

1. Run the provided code to conduct PCA on the *iris* dataset in base R.
2. Render a Scree plot of the PCs.
   - How many PCs should we select?
3. Use your model analysis from Illustrative Example 1 and the provided code to render a scree plot.
   - How many PCs should we select?

# Example 3

1. Use both your base R analysis and h2o analysis to generate scree plots for the PCA of the *Basketball.csv* dataset.
2. How many PCs should we select?
3. Do the results differ across methods?

# (2) Variability Explained Thresholds

- Some typical thresholds for the amount of variance explained are 75% and 90%.
- In other words, keep all PCs that explain at least that much of the variability.
- This information is easily obtained from the analysis object in R.
- *See example code to plot this information.*

# Example 4

1. How many PCs are selected at the 75% and the 90% thresholds?
2. Adjust the code to plot these results.

# (3) Eigenvalue Criterion

- Recall that the sum of the eigenvalues is equal to the number of variables ($p$).
  - These eigenvalues may be greater or less than one.
- Any eigenvalue that is greater than one, *explains over one variable's worth of the variability.*
- **This approach says to keep all PCs whose corresponding eigenvalue is greater than or equal to 1.**
  - Remember, the variance is just the squared standard deviation!

# Example 5

1. Based on the **Eigenvalue Criterion** how many PCs should we keep in the *Basketball.csv* data?

# Comments on Number of PCs

- There is an element of subjectivity in the process of selecting the number of principal components.
- In some cases, the answer may be fairly obvious.
  - In other scenarios it may be less obvious.
- As with any decision in data science, it is important to be able to justify why you made that decision.

# Applications

- Principal Components can be used for statistical or machine learning.
  - Especially useful when there are many variables being considered.
- They may be used in inferential statistics.
  - Very difficult to interpret or explain the results.
- The PC are linear by nature and may fail to capture non-linear relationships in data.

# Illustrative Example 3

- This example shows the process of using PCs in a machine-learning algorithm.
- Run the code and examine the results.
- Is this approach really needed?

# Example 6

1. Repeat the process used in Illustrative Example 3 on the scaled Basketball data.
2. How many PCs did you select?
3. How accurate was this approach?

# Singular Value Decomposition

- You can conduct Singular Value Decomposition (SVD) using base R:

  `prcomp(x,` - *dataset*
  `retx = TRUE,` - *return the rotated values*
  `center = TRUE,` - *centre each variable on zero*
  `scale. = FALSE)` - *scale the variables*

- Will produce similar results to PCA in base R, see the documentation for more details.

# Final Thoughts

- PCA works best with many highly correlated variables.
- This is still very much a *linear* approach to understanding data.
- Principal Component Regression may be applied, but interpretations are quite difficult.
- There are other methods for reducing dimensions and understanding patterns in data.

# Next Lecture

- We will discuss $K$-**Nearest Neighbours**.
- Review distance measures.
- Choosing $k$.
- Hyperparameter tuning.

# Exercise 1

- Import the wine dataset from the *HDclassif* package.
- Normalize your data.
- Split your data into 75/25 train test datasets.
- Conduct Eigenanalysis using both methods shown in these slides using your training data.
- Determine how many PCs are needed.
- Use your PCs combined with other methods you have learned to predict classes in the testing set.
- What do you notice?

# References & Resources


1. Bakker, J. D. (2026) Applied Multivariate Statistics in R
2. Peng R. D. (2020) *Exploratory Data Analysis with R*
3. Timbers T., Campbell T., and Lee M. (2024) *Data science: A first introduction.* Chapman and Hall/CRC
4. Bradley Boehmke B., and Greenwell B. (2020) *Hands-On Machine Learning with R.* Taylor & Francis


- eigen()
- princomp()
- h2o.prcomp()
- h2o